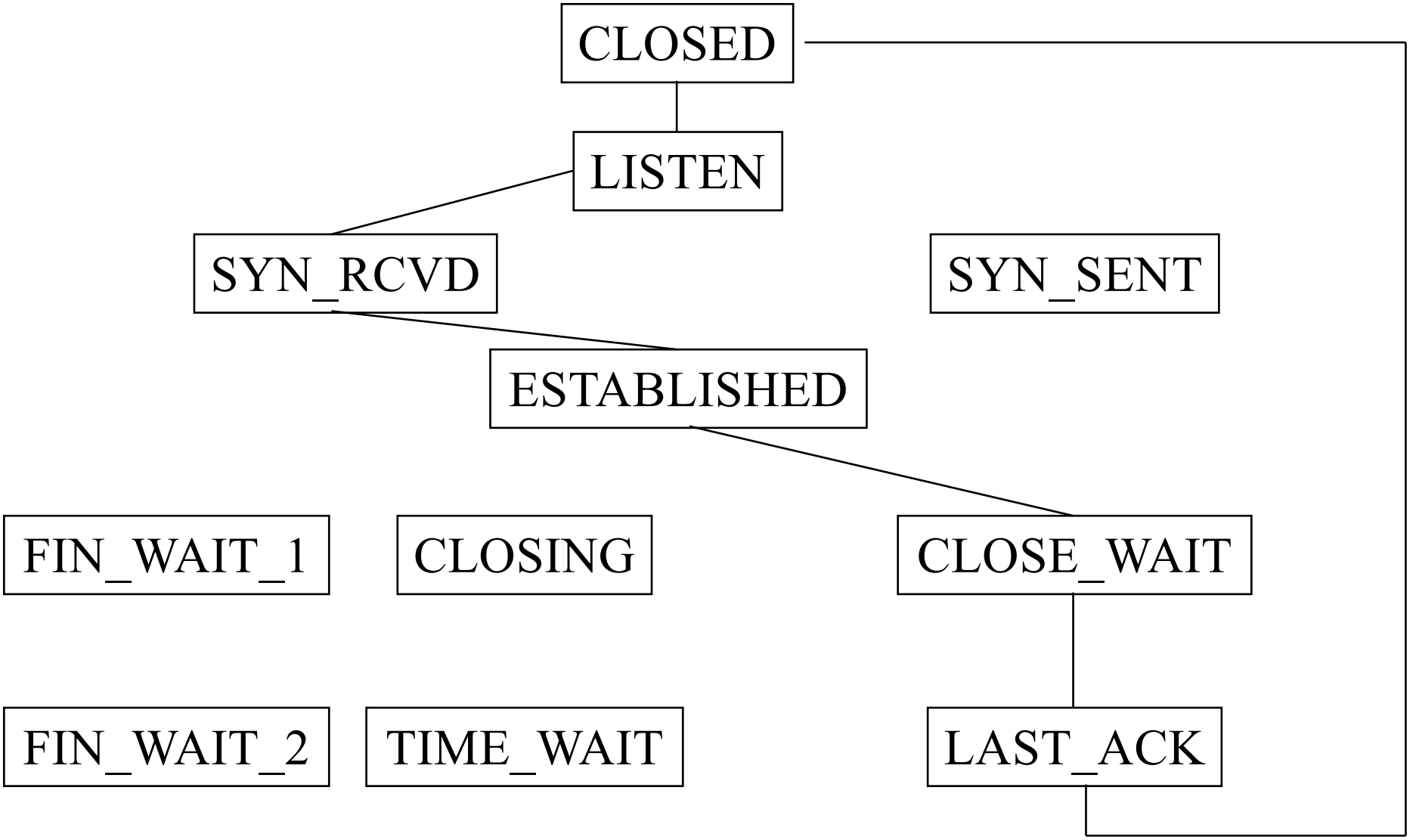
Netzwerkprogrammierung

Vorlesung 4

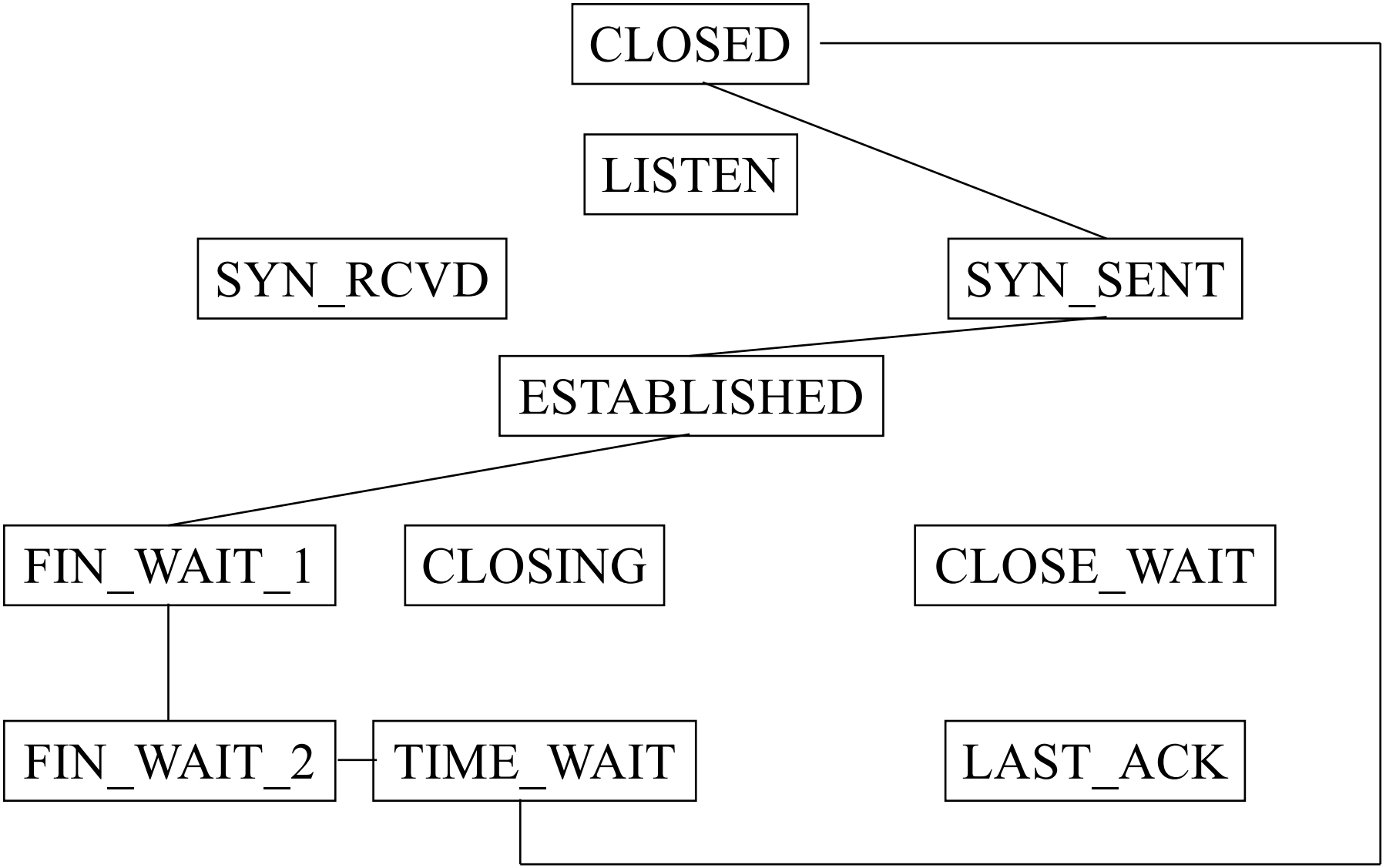
Transmission Control Protocol

- Bytestreamorientiert
- Gesichert
- Flowcontrol
- Congestioncontrol
- Segmentierung
- Singlehoming
- Unicast

TCP State Engine (Server)



TCP State Engine (Client)



Mehrere Verbindungen

- Demultiplexen (select).
- Mehrere Prozesse (fork).
- Mehrere Threads (pthread_create).

select call

- int select(int nfds, fd_set *readfds,
  fd_set *writefds,
  fd_set *exceptfds,
  struct timeval *timeout);
- Gibt die Anzahl der bereiten Filedescriptoren zurück, -1 im Fehlerfall.

FD_* macros

- FD_SET(fd, &fdset);
- FD_CLR(fd, &fdset);
- FD_ISSET(fd, &fdset);
- FD_ZERO(&fdset);

timeval structure

- struct timeval {
  int32_t tv_sec; /* seconds */
  int32_t tv_usec; /* and microseconds */
  };

A better sleep...

- timeout->tv_sec = 0;
  timeout->tv_usec = 100;
  select(0, NULL, NULL, NULL, timeout);
- sleep hat als minimale Zeit 1 Sekunde.

Ein besserer client

- stdin wird an den Server gesendet.
- Vom Server empfangene Nachrichen werden auf stdout ausgegeben.
- Es muß also von zwei Quellen gelesen werden.

Ein bisschen Code...

```
FD_ZERO(&rset);
while(!done) {
    FD_SET(0, &rset);
    FD_SET(fd, &rset);

    Select(fd + 1, &rset, NULL, NULL, NULL);
    if (FD_ISSET(0, &rset)) {
        n = Read(0, (void *) buf, sizeof(buf));
        if (n==0)
            done = 1;
        else
            Send(fd, (const void *) buf, (size_t) n, 0);
    }
    if (FD_ISSET(fd, &rset)) {
        n = Recv(fd, (void *) buf, sizeof(buf), 0);
        if (n == 0)
             done = 1;
        else
           printf("%.*s", (int)n, buf);
    }
}
```

Echo Server

- Mittels select() kann man mehrere Clients gleichzeitig bearbeiten.
- Problem: Ein Systemcall blockiert.
- Problem: send().
- Nicht blockierender send() call für zu komplizierteren Programmen.

Echo Server mit mehreren Prozessen

- Nachdem eine Verbindung akzeptiert wurde, wird pro Client ein Prozess gestartet, der diesen Client bearbeitet.
- Nicht notwendige Resourcen (Filedescriptoren) sollte man freigeben.

fork call

- pid_t fork(void);
  - Gibt 0 im Childprozess zurück, pid des Childprozesses im Parentprozess und -1 im Fehlerfall.

Echo Server mit mehreren Threads

- Nachdem eine Verbindung akzeptiert wurde, wird pro Client ein Thread gestartet, der diesen Client bearbeitet.
- Man muss darauf achten, dass gemeinsame Resourcen geschützt werden.

pthread_create call

- int pthread_create(pthread_t *tid,
  const pthread_attr_t *attr,
  void *(*f) (void *),
  void *arg)
  - *tid: Threadidentifier des neuen Threads
  - *attr: Attribute des neuen Threads, NULL für Defaultwerte.
  - *f: main Funktion des Threads.
  - *arg: Argument, mit dem f aufgerufen wird.

pthread_detach call

- int pthread_detach(pthread_t tid);
  - tid: Threadidentifier des Threads, der detached wird.
- Threads sind ‚joinable' oder ‚detached'

pthread_self

- pthread_t pthread_self(void);
  - Gibt den Threadidentifier des Threads zurück.
- Entspricht getpid bei Prozessen.
- Der tid ist nur eindeuting innerhalb eines Prozesses.

Echo Server, schlecht gemacht

- In main wird nach accept aufgerufen:
  pthread_create(&tid, NULL,
  &handle_connection,
  (void *) cfd);
- In handle_connection wird eine lokale Variable cfd vereinbart und initialisiert mit
  fd = (int)arg;

Echo Server fehlerhaft

- In main wird nach accept aufgerufen:
  pthread_create(&tid, NULL,
  &handle_connection,
  (void *) &cfd);
- In handle_connection wird eine lokale Variable cfd vereinbart und initialisiert mit
  fd = *((int *)arg);

Multithread versus Multiprocess

- Sicherheit
- Resourcen
- Geschwindigkeit
- Skalierbarkeit
- Plattformabhängig.

Das Domain Name System

- Verteiltes System.
- Basisfunktionalität des Internets.
- Hierarchisches System.
- Benutzt sowohl TCP als auch UDP.
- Client/Server Architektur.

Resource Records

- A: IP-address (V4)
- AAAA: IP-address (V6)
- CNAME: Canonical Name
- PTR: Pointer Record
- HINFO: Host Info
- MX: Mail Exchange

Der Resolver

- Programme benutzen Bibliotheksfunktionen um Namensauflösungen durchzuführen.
- Der Resolver benutzt lokale Daten (/etc/hosts).
- Der Resolver benutzt konfigurierte Nameserver (/etc/resolv.conf).
- Teil des Betriebssystems.
- Realisiert keinen Cache.

Probleme mit DNS

- Weder Clients noch Server authentifizieren sich.
- DOS und DDOS Attacken auf Root Server.
- Kein gesicherter Transport für Abfragen.
- Limitierte Nachrichtenlänge.
- Verfügbarkeit
- Cache

gethostbyname()

- struct hostent *gethostbyname
  (const char *hostname)
  - hostname: Name, dessen IP Adresse gesucht ist
  - Im Fehlerfall wird NULL zurückgegeben, sonst ein Pointer auf eine hostent Struktur.
  - Im Fehlerfall ist h_errno gesetzt. hstrserror(h_errno) gibt eine Fehlerbeschreibung.

gethostbyname2()

- struct hostent *gethostbyname2
  (const char *hostname, int family)
  - hostname: Name, dessen IP-Adresse gesucht wird.
  - family: AF_INET oder AF_INET6
  - Fehlerbehandlung wie bei gethostbyname

hostent structure

```
struct  hostent {
  char    *h_name;        /* official name of host */
  char    **h_aliases;    /* alias list */
  int     h_addrtype;     /* host address type */
  int     h_length;       /* length of address */
  char    **h_addr_list;  /* list of addresses from
                             name server */
};
```

getservbyname() call

- struct servent *getservbyname
  (const char *servname,
  const char *protoname)
  - servname: Name des Dienstes: ‚echo‘, ‚discard‘
  - protoname: Name des Protokolls: ‚tcp‘, ‚udp‘ oder NULL.

struct servent

- ```
  struct  servent {
    char    *s_name;        /* official service name */
    char    **s_aliases;    /* alias list */
    int     s_port;         /* port # */
    char    *s_proto;       /* protocol to use */
  };
  ```
  ```

Beispiel: Ein besserer Client

- Usage: client hostname service
- Übersetzt den hostname und service.
- Benutzt alle IPv4 Adressen.
- Wie werden IP-Adressen behandelt?
- Wie werden Portnummern behandelt?

Wie behandelt man die Adressen?

```
for (pptr = (struct in_addr **) hp->h_addr_list; *pptr != NULL; pptr++) {
        fd  = socket(AF_INET, SOCK_STREAM, 0);
        memset(&server_addr, 0, sizeof(server_addr));
        server_addr.sin_len     = sizeof(server_addr);
        server_addr.sin_family  = AF_INET;
        server_addr.sin_port    = sp->s_port;
        memcpy((void *) &server_addr.sin_addr, (const void *) *pptr, sizeof(struct in_addr));
        if (connect(fd, (const struct sockaddr *) &server_addr, sizeof(server_addr)) == 0)
                break;
        printf("connect failed.\n");
        close(fd);
}
```